

GUIDE DE COMPRÉHENSION

Régression Logistique & XGBoost

Comprendre les modèles, leurs paramètres, et interpréter les résultats

Document d'accompagnement — Chapitres 5.2 et 5.4
Projet : Détection de fraude bancaire par Machine Learning

Introduction

Ce document explique en détail les deux modèles dont tu as la responsabilité dans le mémoire : la Régression Logistique (section 5.2) et XGBoost (section 5.4). L'objectif est double : comprendre le principe de chaque algorithme et le rôle exact de chaque paramètre utilisé dans ton notebook, et savoir lire les résultats obtenus (matrice de confusion, precision, recall, F1-score, AUC-ROC) pour pouvoir les commenter à l'oral comme à l'écrit.

Une fois que tu auras exécuté le notebook et obtenu tes propres chiffres, il suffira de les reporter dans les tableaux et graphiques ci-dessous décrits pour rédiger le texte définitif des sections 5.2 et 5.4 du rapport.

1. Régression Logistique

1.1 Principe de l'algorithme

La Régression Logistique est un modèle de classification linéaire. Contrairement à la régression linéaire classique qui prédit une valeur continue, elle prédit une probabilité comprise entre 0 et 1 grâce à la fonction sigmoïde :

$$P(\text{Fraude}) = 1 / (1 + e^{-(\beta_0 + \beta_1 \cdot X_1 + \beta_2 \cdot X_2 + \dots + \beta_n \cdot X_n)})$$

Chaque variable X_i (V_1 à V_{28} , Amount, Time) est multipliée par un coefficient β_i appris pendant l'entraînement. Un coefficient positif signifie que la variable augmente la probabilité de fraude ; un coefficient négatif signifie qu'elle la diminue. Le modèle applique ensuite un seuil (par défaut 0.5) : si la probabilité prédite dépasse ce seuil, la transaction est classée comme frauduleuse.

C'est un modèle volontairement simple, utilisé comme baseline : il sert de point de comparaison pour juger si des modèles plus complexes (Random Forest, XGBoost) apportent un gain réel de performance.

1.2 Paramètres utilisés dans le code

Paramètre	Valeur utilisée	Signification
max_iter=1000	Nombre maximal d'itérations	L'algorithme d'optimisation (descente de gradient) ajuste les coefficients par itérations successives. Avec 30 variables, 1000 itérations garantissent la convergence (éviter l'avertissement "non convergé").
random_state=42	Graine aléatoire fixe	Garantit que les résultats sont reproductibles : en relançant le code, on obtient exactement les mêmes coefficients.
penalty='l2'	Régularisation Ridge (L2)	Pénalise les coefficients trop élevés pour éviter le sur-apprentissage (overfitting). L2 réduit l'amplitude des coefficients sans les annuler complètement (contrairement à L1 qui peut les mettre à zéro).

Paramètre	Valeur utilisée	Signification
Entraînement sur X_train_sm / y_train_sm	Données rééquilibrées par SMOTE	Le modèle apprend sur un jeu d'entraînement équilibré à 50/50, ce qui l'empêche de simplement prédire "normal" tout le temps (ce qu'il ferait sinon, vu que 99,83 % des transactions sont normales).
Évaluation sur X_test / y_test	Données de test originales, non modifiées	Essentiel : le test doit refléter la réalité (0,17 % de fraude), sinon les métriques seraient artificiellement optimistes. C'est le principe anti-fuite de données (data leakage) déjà appliqué dans la section 4.3.

1.3 Le seuil de décision (threshold)

Par défaut, une transaction est classée frauduleuse si sa probabilité prédite dépasse 0.5. Mais ce seuil n'a rien d'obligatoire : dans le notebook, plusieurs seuils sont testés (0.5, 0.3, 0.2, 0.1) pour observer l'effet suivant :

- Seuil plus bas (ex. 0.1) → le modèle devient plus "méfiant" : il détecte plus de fraudes réelles (recall plus élevé) mais déclenche aussi plus de fausses alertes (precision plus faible).
- Seuil plus haut (ex. 0.5) → le modèle est plus "sélectif" : moins de fausses alertes, mais certaines fraudes réelles passent inaperçues.

Dans un contexte bancaire, manquer une fraude coûte généralement plus cher qu'une fausse alerte (qui ne fait que déclencher une vérification manuelle) : c'est l'argument à utiliser dans le rapport pour justifier de privilégier un seuil plus bas que 0.5 si le recall progresse nettement sans effondrer la precision.

1.4 Coefficients et importance des variables

Le graphique des "Top 10 coefficients" montre quelles variables poussent le plus vers la prédiction de fraude (coefficients positifs, en général en lien avec V4, V11 dans l'EDA) ou au contraire vers la prédiction de transaction normale (coefficients négatifs, en lien avec V17, V14, V12, V10, V16 qui étaient déjà les variables les plus corrélées négativement avec Class dans l'analyse exploratoire du chapitre 3). Il est intéressant de vérifier dans le rapport si ces coefficients confirment bien les corrélations observées en EDA — cela renforce la cohérence de l'ensemble du mémoire.

2. XGBoost

2.1 Principe de l'algorithme

XGBoost (eXtreme Gradient Boosting) est un modèle d'ensemble basé sur le boosting séquentiel d'arbres de décision. Contrairement au Random Forest (bagging), où les arbres sont construits indépendamment puis moyennés, XGBoost construit ses arbres un par un, chaque nouvel arbre étant entraîné pour corriger les erreurs commises par les arbres précédents.

Concrètement : le premier arbre fait des prédictions imparfaites ; le deuxième arbre se concentre sur les cas où le premier arbre s'est trompé (les résidus) ; et ainsi de suite jusqu'à n_estimators arbres. La prédiction finale est une somme pondérée des prédictions de tous les arbres. Cette approche séquentielle rend XGBoost généralement plus performant que le bagging sur des données tabulaires, au prix d'un temps d'entraînement plus long et d'un risque de sur-apprentissage plus élevé s'il n'est pas bien réglé.

2.2 Paramètres utilisés dans le code

Paramètre	Valeur utilisée	Signification
n_estimators=300	Nombre d'arbres construits séquentiellement	Plus il y a d'arbres, plus le modèle peut apprendre de motifs fins, mais au-delà d'un certain point le gain devient marginal et le risque de sur-apprentissage augmente. 300 est une valeur courante de départ.

Paramètre	Valeur utilisée	Signification
max_depth=6	Profondeur maximale de chaque arbre	Contrôle la complexité de chaque arbre individuel. Une profondeur trop grande favorise le sur-apprentissage (l'arbre "mémorise" les données) ; trop faible, le modèle est trop simple (sous-apprentissage).
learning_rate=0.1	Taux d'apprentissage (aussi noté eta)	Pondère la contribution de chaque nouvel arbre à la prédiction finale. Une valeur faible (0.1) rend l'apprentissage plus progressif et stable, généralement au prix d'un nombre d'arbres plus élevé nécessaire pour converger.
eval_metric='logloss'	Métrique de suivi de l'entraînement	La logloss pénalise fortement les prédictions confiantes mais fausses ; elle est adaptée à un problème de classification probabiliste comme celui-ci.
random_state=42	Graine aléatoire fixe	Reproductibilité des résultats d'un run à l'autre.
n_jobs=-1	Nombre de coeurs CPU utilisés	-1 signifie "utiliser tous les coeurs disponibles" pour accélérer l'entraînement (paramètre de performance, sans effet sur les résultats).
scale_pos_weight (variante)	Poids relatif de la classe minoritaire	Alternative native à SMOTE : au lieu de dupliquer/générer des exemples de fraude, ce paramètre indique au modèle de pénaliser davantage les erreurs sur la classe minoritaire. Calculé ici comme le ratio (nombre de transactions normales / nombre de fraudes) dans le train set.

2.3 Deux variantes comparées

Le notebook entraîne XGBoost de deux façons différentes, ce qui permet une discussion intéressante dans le rapport :

- XGBoost + SMOTE : entraîné sur les mêmes données rééquilibrées que la Régression Logistique — permet une comparaison directe et équitable entre les deux modèles.
- XGBoost + scale_pos_weight : entraîné sur les données originales déséquilibrées, sans SMOTE — permet de vérifier si l'approche native de XGBoost égale, dépasse, ou est en retrait par rapport à SMOTE.

Si les deux variantes donnent des résultats proches, cela confirme que scale_pos_weight est une alternative valable et moins coûteuse en calcul (pas besoin de générer 227 057 exemples synthétiques). Si SMOTE fait significativement mieux, cela justifie de le garder comme technique principale dans le mémoire.

2.4 Importance des variables (feature importance)

XGBoost calcule une importance pour chaque variable, basée sur la fréquence et la qualité de son utilisation dans les décisions de tous les arbres (par défaut, le gain moyen qu'elle apporte). Le graphique "Top 10 importances" doit être comparé à la fois aux corrélations de l'EDA (chapitre 3) et aux coefficients de la Régression Logistique (section 1.4 ci-dessus) : si V17, V14, V12, V10 et V16 ressortent également ici, c'est un signal fort de cohérence entre les trois analyses (EDA, modèle linéaire, modèle non-linéaire) — un bon argument pour la section 6.3 (Analyse et interprétation des performances).

3. Glossaire des métriques — comment lire les résultats

Ce glossaire s'applique aux deux modèles ; il te permet d'interpréter directement ce qu'affiche 'classification_report' et 'confusion_matrix' dans ton notebook.

3.1 La matrice de confusion

Pour un problème à deux classes (Normal / Fraude), la matrice de confusion croise les prédictions du modèle avec la réalité :

	Prédit : Normal	Prédit : Fraude
Réel : Normal	Vrai Négatif (VN) transaction normale, bien classée	Faux Positif (FP) fausse alerte : normale classée fraude
Réel : Fraude	Faux Négatif (FN) fraude non détectée — le cas le plus grave	Vrai Positif (VP) fraude bien détectée

Dans ce projet, le Faux Négatif (fraude non détectée) est l'erreur la plus coûteuse : c'est une perte financière directe pour la banque ou le client. Le Faux Positif (fausse alerte) a un coût plus faible (vérification manuelle, gêne pour le client), ce qui justifie souvent de privilégier le recall sur la precision — ou du moins de trouver un compromis raisonné.

3.2 Precision, Recall, F1-Score

Paramètre	Valeur utilisée	Signification
Precision	$VP / (VP + FP)$	Parmi toutes les transactions que le modèle a signalées comme fraude, quelle proportion l'est vraiment ? Une precision faible signifie beaucoup de fausses alertes.
Recall (rappel)	$VP / (VP + FN)$	Parmi toutes les fraudes réelles, quelle proportion le modèle a-t-il réussi à détecter ? Un recall faible signifie que beaucoup de fraudes passent inaperçues — critique dans ce contexte.
F1-Score	$2 \times (Precision \times Recall) / (Precision + Recall)$	Moyenne harmonique des deux : un résumé utile quand on veut un seul chiffre, mais il ne remplace pas l'analyse séparée de precision et recall, surtout avec des classes très déséquilibrées.

3.3 AUC-ROC et courbes

La courbe ROC (Receiver Operating Characteristic) trace le taux de vrais positifs contre le taux de faux positifs pour tous les seuils de décision possibles, de 0 à 1. L'AUC-ROC (Area Under Curve) résume cette courbe en un seul chiffre entre 0.5 (modèle aussi bon qu'un tirage aléatoire) et 1 (modèle parfait). C'est une métrique globale, indépendante du choix d'un seuil précis — utile pour comparer des modèles entre eux.

La courbe Precision-Recall est souvent plus informative que la ROC lorsque les classes sont très déséquilibrées (comme ici, avec 0,17 % de fraude) : elle montre directement le compromis entre precision et recall à mesure que le seuil varie, sans être "gonflée" artificiellement par le grand nombre de vrais négatifs faciles à classer.

3.4 Lire le tableau comparatif final

Le tableau comparatif produit en fin de notebook (Régression Logistique vs XGBoost SMOTE vs XGBoost scale_pos_weight) permet de répondre directement à l'une des questions classiques de maintenance : "Pourquoi XGBoost performe-t-il mieux (ou moins bien) que la Régression Logistique ?". En général, si XGBoost a une AUC-ROC et un F1-Score plus élevés, c'est parce qu'il capture des relations non-linéaires et des interactions entre variables (par exemple entre V17 et V14) que la Régression Logistique, purement linéaire, ne peut pas modéliser.

4. Passer du notebook au texte du rapport

Une fois le notebook exécuté avec tes vrais résultats, voici comment structurer le texte des sections 5.2 et 5.4 (qui remplacera les placeholders en italique gris) :

- Rappeler brièvement le principe du modèle (une ou deux phrases, voir sections 1.1 / 2.1 ci-dessus).
- Lister les hyperparamètres retenus et justifier chacun en une phrase (voir les tableaux de paramètres ci-dessus, à réutiliser presque tels quels).
- Présenter les résultats chiffrés : matrice de confusion, precision, recall, F1-score, AUC-ROC (insérer le Tableau 3 déjà prévu dans la liste des tableaux du rapport).

- Interpréter : quelle erreur domine (FP ou FN) ? Le seuil par défaut est-il adapté ou faut-il le déplacer ? Quelles variables dominent (cohérence avec l'EDA) ?
- Pour XGBoost, ajouter la comparaison SMOTE vs scale_pos_weight, et conclure sur l'approche retenue pour le reste du mémoire (chapitre 6).

Envoie-moi les résultats obtenus (copier-coller les sorties des cellules, ou les chiffres clés) et je rédigerai directement le texte académique définitif des sections 5.2 et 5.4 dans le fichier Word, dans le même style que les chapitres déjà finalisés.